

Day 1 · 8/4

企业需要怎样的 Agent 底座？

大厂技术路线与 Agent OS 架构拆解

从行业格局 → 自建必要性 → 架构与实现 → 开发者的核心竞争力

企业级 AI 编程实战营 · 系列直播（一）

今晚的路线

四段，从行业看到自己



一句话：先看清企业在这一层要什么，再看这套底座该怎么搭、AI 能帮到什么程度，最后回到你我该练什么。

提示：今晚全程讲设计与取舍，涉及的完整代码与实现在训练营正课展开。

2026：企业级 Agent 平台成了必争之地

- 巨头几乎同时下场：华为 AgentArts、阿里悟空、字节 Coze / HiAgent、百度 AgentBuilder、腾讯元器。
- 共同点：都不再只卖模型，而是卖“底座 + 生态”——把 Agent 的运行、编排、治理打包成平台。
- 分歧点：云厂商从基础设施往上做，平台方从应用生态往下做，赌的东西不一样。

为什么值得先看它：大厂的投入方向，基本等于这一层未来三年的技术标准走向；看清路线，才知道自己该建什么、学什么。

六家的位置：从开源框架到托管底座

华为 AgentArts

云 + 算力 + 模型，从基础设施往上做

阿里 AgentScope

开源多智能体框架：框架 + Runtime + Studio

阿里 悟空

企业级 Agent 平台，规模化推进中

AWS AgentCore

托管底座：Runtime / Gateway / Identity / 可观测

字节 Coze · HiAgent

轻集成、快部署，做应用生态

百度 AgentBuilder · 腾讯 元器

各自生态内的 Agent 构建平台

读法：越靠“基础设施/托管”的强调可控与免运维；越靠“框架/生态”的强调低门槛与场景数量。

AgentScope 与 AgentCore: 一个开源框架, 一个托管底座

阿里 AgentScope

- 开源框架, 代码和部署在你手上
- 框架 + Runtime (沙箱/K8s) + Studio (监控)
- 覆盖开发、部署、监控全流程
- 面向开发者: 多智能体协作怎么写

AWS AgentCore

- 云托管平台, 不用自己管基础设施
- Runtime 会话隔离 · Gateway 用 MCP 把 API 变工具
- Identity 身份委托 · Observability / 评估
- 面向企业: 任意框架、任意模型都能跑

对比出的结论: 两者要解决的问题高度重叠 (运行、工具、记忆、可观测), 差别在交付方式——开源自持 vs 云上托管。这也说明: 底座该有哪些能力, 业界其实在收敛; 叫什么、谁来托管, 还没定。

Agent 平台、Agent OS、Agent 框架 —— 叫法之争

Agent 平台 / 框架

- 偏“怎么把 Agent 开发出来”
- 编排、模板、低代码、多智能体协作
- 面向开发者的生产力工具

Agent OS / 底座

- 偏“一群 Agent 怎么被管起来”
- 生命周期、渠道、记忆、权限、审计
- 面向企业的运行与治理底座

我的看法：做的事情其实很像，叫法不一样；各自的定位有差别，有重叠也有分工。这一层还没有统一标准，属于百花齐放的阶段。

所以别纠结名字：看它到底解决了哪几件事，比看它叫什么有用。

两种打法，代表两种假设

自下而上：做底座

- 从算力、模型、运行时往上做
- 强调私有化、可控、行业深度
- 假设：企业最终要把 Agent 跑在自己手里

自上而下：做生态

- 从应用、编排、模板往下做
- 强调低门槛、快部署、场景多
- 假设：先把量做起来，标准自然归我

对企业的启示：这两条路都不解决一件事——你自己的差异化流程、私有数据和治理要求。这正是自建底座的起点。

有了这么多平台，为什么还要自建？

- 行业已经不再是“自建 vs 采购”的二选一——现在的共识是混合：通用流程用平台，差异化流程自己掌控。
- 判断标准很实在：这个 Agent 本身算不算你的竞争力？算，就得自己握住编排、数据与模型选择权。
- 另一个现实约束：规模。调用量与复杂度上到一定程度，自建的三年总成本反而更划算。

所以问题不是“建不建”，而是：哪一部分必须自己建、建成什么样。下面三页说清这件事。

买不来的四件事

这就是自建底座的必要性

数据主权

私有数据、知识库、会话记录不出企业；模型可换、数据不动

审计与合规

每一次工具调用、每一条决策链路可追溯，满足内部与监管要求

多 Agent 治理

几十个 Agent 的注册、权限、配额、生命周期，需要统一管

模型自由

多 Provider 抽象，随时切换/混用，不被单一厂商锁定

一句话：平台能给你能力，给不了你控制权。控制权只能自己建。

什么交给平台，什么自己掌控

交给平台（通用流程）

- 标准问答、文档摘要、通用客服
- 一次性、低价值、无差异化的场景
- 验证期：先用平台快速试对不对

自己掌控（差异化）

- 与核心业务耦合的编排与工具
- 涉及私有数据、权限、审计的链路
- 需要长期演化、沉淀为资产的部分

实践建议：用平台跑通场景验证，用自建底座承载长期资产。两者不是对立，是分工——但底座必须在自己手上。

Agent OS: 企业的 Agent 统一底座

定义：运行和管理 AI Agent 的底座系统。向上给各类 Agent（运维、客服、HR、销售...）提供统一运行环境，向下接入模型、渠道、工具、记忆、身份与审计。

agent runtime

让“一个”agent 跑起来：LLM 调用、工具执行、上下文、循环控制。

Agent OS

让“一群”agent 在企业里被管起来：生命周期、统一接入、统一记忆、多租户与审计。

类比：runtime 像单个进程的执行环境，Agent OS 像管理一群进程、调度资源、提供共享服务的那层。

缺任何一件，就不叫底座

① Agent 生命周期

注册、启动、监控、销毁；每个 Agent 有独立 prompt / 模型 / 工具 / 渠道 / 记忆

② 统一对外渠道

IM（企微 / 飞书 / 钉钉 / Slack）、邮件、Web、HTTP API 共用一套渠道层

③ 统一对内接入

LLM Provider、工具（MCP / 插件）、企业 IT 系统、知识库，共享一套接入层

④ 统一记忆

跨 Session 长期记忆、可复用 Skill、跨 Agent 的知识沉淀

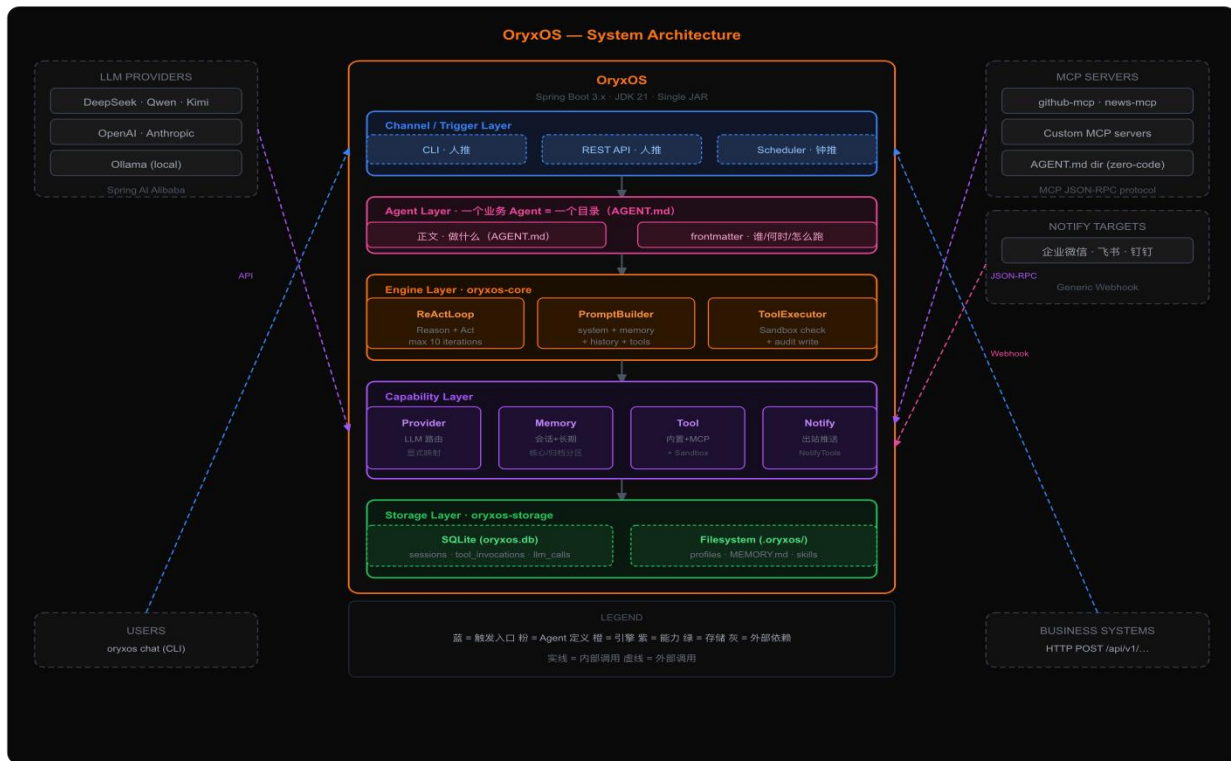
⑤ Tool 调用与沙箱

Function Calling 驱动工具执行，沙箱内运行，守住安全边界

整体架构

一张图看清底座的层次

1. Provider
2. Memory
3. Tool
4. React
5. SandBox
6. Notify
7. 安全审计
8. 分布式



LLM Provider 抽象：模型自由的前提

- 把“调模型”这件事收成一个统一接口：文本、流式、Function Calling、多模态各自有稳定契约。
- 多 Provider 并存：OpenAI 兼容、国产模型、私有化部署模型，同一套上层代码都能跑。
- 关键设计：能力探测（这个模型支不支持 tools / vision）+ 失败降级 + 超时与重试策略。
- 落到工程上：Provider 是插件，新增一个模型厂商不改上层业务代码。

为什么它排第一：模型迭代速度远快于业务代码。抽象没做好，每换一次模型就要动一遍全身。

ReAct 推理循环：Agent 的心跳



- 工程上要管三件事：最大轮数、上下文长度控制、失败/歧义时的退出策略。
- 可观测很重要：每一轮的 Reason / Act / Observe 都要留痕，否则线上出问题无从查起。

提示：ReAct 的实现细节与代码讲解，是正课里单独一节。

Tool 调用：Function Calling 与 MCP

Function Calling

- 把工具描述成 schema 交给模型
- 模型决定调哪个、传什么参数
- 执行结果回灌，进入下一轮推理

MCP（标准化接入）

- 工具/数据源按统一协议暴露
- 一次接入，多个 Agent 复用
- 生态在快速变厚，值得押注

底座该提供什么：工具注册与发现、参数校验、权限与配额、调用留痕、失败重试——这些不做，Agent 一上生产就出事。

安全底线：工具在沙箱里执行；密钥统一由底座注入，不进 prompt、不落日志。

Memory: 让 Agent 有连续性

短期 / 会话

当前对话上下文，受窗口限制，需要裁剪与摘要

长期 / 用户

跨 Session 的偏好、事实、历史结论，可检索

知识 / 检索

企业知识库、文档，RAG 召回后进上下文

技能 / 复用

沉淀成可复用的 Skill 模板，跨 Agent 共享

工程难点：不是“存下来”，而是“什么时候取、取多少、怎么不污染上下文”。记忆做过头，比不做更糟。

WebService 与渠道接入：让 Agent 能被用到

- 对外：IM（企微 / 飞书 / 钉钉 / Slack）、邮件、Web、HTTP API —— 所有 Agent 共用一套渠道层。
- 渠道层要抹平差异：消息格式、富文本、文件、@ 与线程、限流与重试，都在这一层收口。
- 对内：统一 HTTP / gRPC 接口，供业务系统直接调用 Agent 能力。
- 配套：会话路由（谁的消息给哪个 Agent）、身份映射（企业账号 ↔ Agent 用户）。

经验：很多 Agent 项目死在这一层——模型效果不错，但接不进企业现有的沟通与系统流程，最后没人用。

插件化：让底座能长期演化

Provider 插件

换模型不改上层

Tool 插件

加工具不动核心

Memory / 存储插件

换存储只加实现

- 做法：接口定义在核心，具体实现放插件目录，用 SPI / 注册表加载。
- 收益：底座核心保持小而稳；能力扩展交给插件，社区/团队可以并行贡献。

判断标准：新增一个模型/工具，需要改核心代码吗？需要，说明抽象没做对。

多租户、权限、沙箱、审计：上生产的门槛

多租户隔离

不同部门/业务的 Agent、数据、配额互不干扰

权限与配额

谁能调哪个工具、每天多少次、超了怎么办

Tool 沙箱

工具执行有资源与网络边界，出错不影响底座

全链路审计

每次调用留痕：谁、什么时候、调了什么、结果如何

这一段决定“能不能用”：demo 阶段没人关心这些；真正进企业，第一轮评审问的就是它们。

关键问题

这么大的项目，AI 能帮到什么程度？

- 先说结论：这类底座的核心代码，绝大部分可以由 AI 写出来——前提是你把“怎么写”设计好。
- AI 不擅长的是“在模糊中做架构决策”，擅长的是“在清晰约束下高强度产出”。
- 所以关键不在提示词技巧，而在三件事：契约先行、分块交付、判官可跑。

下面三页：把这三件事讲清楚——这也是训练营里贯穿 11 周的主线方法。

做法 ①

契约先行：先定领域模型和接口

- 先把跨模块的东西钉死：核心领域模型（一个 Agent / 一次会话 / 一次工具调用长什么样）+ 各层接口。
- 契约定完再分模块。否则每块各自定义，最后拼不到一起——这是 AI 大项目最常见的翻车方式。
- 契约是唯一真源：改接口先改契约，再让实现跟上，不允许反向漂移。

分块交付：每块都有明确输入和验收

输入

这块做什么 · 交互 · 接口 · 参数

约束

不许碰什么 · 拿不准就停

验收

可自动判定的断言 + 必须有的测试

- 一次只交给 AI 一块，跑到验收全绿再进下一块——过程可控、出错可定位。
- 人的位置：定义输入与验收、审关键边界；执行中段可以不在场。

效果：大项目不再是“一口气写完然后调一个月”，而是一块块稳定长出来。

判官可跑：没有验证，AI 只会自信地错

判官是什么

- 自动化测试、类型检查、编译、CI
- 每一轮都能回答“做对了没”
- 有判官，AI 才能自己收敛

最要防的：判官被收买

- AI 为了变绿会改测试、删测试
- 对策：测试不可改写进规则
- 重的验证交 CI —— AI 够不着

这就是 Harness：代码规范 + 安全红线 + 验收判官。它决定你返工几次——明天第二场会专门讲透。

提示：Agent OS 这类项目的验收，重点在工具调用正确性、权限边界、会话一致性。

Agent 底座未来三年会怎么走

- 协议会收敛：工具与数据接入逐步标准化（MCP 这类协议会像当年的 JDBC/ODBC）。
- 底座会下沉成基础设施：像今天的消息队列、注册中心一样，成为企业标配的一层。
- 竞争点从“能不能跑”变成“治理与成本”：审计、权限、配额、可观测、多模型调度。
- 开源会占住相当一块：企业要可控，就一定需要能自己审、自己改的底座实现。

机会在哪：标准还没定死、底座还没成型的窗口期，正是个人和小团队能切进去的时候。

这条技术路线上，你该长什么能力

- 岗位在变：招聘要求里“会用 AI”已经是默认项，真正被考察的是能不能交付一个系统。
- 工作内容在变：写代码的比重下降，理解需求、定义契约、设计验证的比重上升。
- 评估方式在变：面试与晋升越来越看“你独立扛过什么”，而不是“你写了多少行”。

所以问题变成：你手上有没有一个能讲清楚、扛得住追问的系统级作品？

什么在贬值，什么会留下

正在贬值

- 把清晰需求翻译成代码的熟练度
- 记住 API 用法、框架配置细节
- 只在单点上“会用某个工具”

更值钱了

- 把模糊问题拆成清晰规格
- 架构判断与技术选型（对齐场景）
- 设计验证：怎么证明它是对的
- 对结果负责的能力与信誉

一句话：AI 替掉的是“执行”，留下的是“定义、判断、担责”——恰好都是老工程师积累最多的东西。

四步，按顺序补

① 规格能力

把模糊需求写成能验证的规格（What / 边界 / 验收）

② 架构判断

会做选型对照：为什么用这个不用那个

③ 验证设计

能为关键失败模式设计测试，而不是追覆盖率

④ 驾驭方法

契约先行 + 分块交付 + 判官可跑，把 AI 用成产能

最有效的练法：找一个真项目，从头到尾扛一遍。四种能力只有在真实约束里才长得出来。

为什么建议做一个大项目

它同时给你三样东西

技术能力

架构、取舍、验证——只有完整交付过才真懂

可讲的作品

面试与评审时，能撑住一层层追问的东西

开源身份

公开的贡献记录，是最难伪造的履历

三者互相放大：能力让你做得出来，作品让别人看得见，开源身份让它长期有效。缺一个，另两个都打折。

现实建议：别从零想一个“完美项目”，选一个真实存在需求、能持续演化的方向。

五句话

- 大厂在这一层同时下注，说明它正在成为企业标配的基础设施。
- 平台能给能力，给不了控制权——差异化流程、私有数据、治理必须自己建。
- 一个底座要解决五件事：生命周期、对外渠道、对内接入、统一记忆、工具与沙箱。
- 用 AI 做这类大项目，靠的是契约先行 + 分块交付 + 判官可跑，不是提示词技巧。
- AI 替掉执行，留下定义、判断和担责——把能力往这三样上长。

明天 (8/5)

一个人用 AI 写一个分布式组件

- 案例：When —— 一个分布式延时投递组件（多层时间轮、副本、10 秒故障切换）。
- 讲清 AI 真能写的部分，和它写不了的部分——边界在哪。
- Spec-Kit 四阶段（/specify → /plan → /tasks → /implement）怎么真正用起来。
- Loop 编程与 Harness：为什么它们直接决定你返工几次。
- 最后回到 OPC：一个人 + AI 的产能边界到底在哪。

同一时间：明晚 8 点，欢迎准时来。

谢谢

Agent 底座这一层，标准还没定死 —— 这正是切进去的时候